

HOMEPEdia — Plan de soutenance & script oral

Projet **T-DAT-902** (Big Data) · groupe **PAR_5** · durée **20 min** + Q&A. Format : **2 parties de 10 min**.
Ce document = plan **slide par slide** : ce qu'on **affiche** + ce qu'on **dit** + le **timing**. À recopier dans PowerPoint / Google Slides.

Structure d'ensemble

Partie	Slide	Durée
1 — Le projet & sa fabrication (10 min)	Slide 1 : Problématique	0:50
	Slide 2 : La solution	1:15
	Slide 3 : Les sources de données	1:30
	Slide 4 : Architecture & traitement (médaillon + Airflow)	1:45
	Slide 5 : Déploiement / infra prod	1:30
	Slide 6 : Gouvernance & qualité des données	1:30
	Slide 7 : Stack technique & choix	1:30
2 — Le métier & la démo (10 min)	Slide 8 : Zoom métier — le score	1:45
	Slide 9 : Zoom métier — les règles justifiées	1:30
	Slide 10 : Démo live	3:30
	Slide 11 : Bilan & difficultés	1:15
	Slide 12 : Questions	2:00

Répartition : Partie 1 = intervenant(s) 1, Partie 2 = intervenant(s) 2 (idéal si vous êtes 2). À plus, sous-
découpez chaque partie. **Répétez chronomètre en main** : viser ~9 min par partie pour garder une
marge.

PARTIE 1 — Le projet & sa fabrication (10 min)

Slide 1 — Problématique (titre + accroche) (0:50)

À l'écran - HOMEPEdia — Où acheter en France ? - Plateforme Big Data d'aide à la décision immobilière
- Groupe PAR_5 · Epitech T-DAT-902 · MSc Pro Promotion 2026 - Une phrase-problème : « *Le problème
n'est pas le manque de données, mais leur lecture.* »

À l'oral

« Bonjour, nous sommes le groupe PAR_5. On vous présente HOMEPEDIA, une plateforme Big Data qui aide un jeune actif à décider **où acheter un bien en France**. Le constat de départ : les données immobilières ouvertes existent — DVF, INSEE, SNCF — mais elles sont brutes et éparpillées. Un prix au m² isolé ne dit pas si un quartier est **abordable au regard des revenus**, bien **desservi, équipé**, ou peuplé de **gens comme moi**. Notre but : croiser ces sources et en faire une **décision** lisible. »

Slide 2 — La solution HOMEPEDIA (1:15)

À l'écran - Une web-app : carte interactive + tableaux de bord. - 5 vues : **Cartographe** (score personnalisé), **Fiche commune**, **Statistiques** (7 graphiques), **Tableau** comparatif, **Analyse** textuelle. - (capture de l'accueil / carte de France)

À l'oral

« Concrètement, c'est une application web. Le cœur, c'est une **carte de France** colorée par un **score de correspondance personnalisé** : l'utilisateur règle l'importance qu'il donne au budget, au transport, aux services... et la carte se recolore à son image. On complète avec des fiches par commune, des statistiques et une analyse rédigée automatiquement. On vous fera la démo en 2^e partie. »

Slide 3 — Les sources de données (1:30)

À l'écran

Domaine	Source	Ce qu'on en tire
Prix & abordabilité	DVF (Etalab) × FiLoSoFi (INSEE)	prix médian m ² , années de revenu
Démographie	Recensement INSEE	pyramide des âges
Transport	Gares SNCF	desserte, distance à la gare
Services	Équipements BPE (INSEE)	santé, commerces, écoles...

À l'oral

« On agrège **quatre domaines**, tous en données publiques françaises. Le prix vient de **DVF** — les transactions réelles — qu'on croise avec les revenus **FiLoSoFi** pour mesurer l'**abordabilité**. On ajoute la démographie du recensement, la desserte via les **gares SNCF**, et les équipements via la base **BPE** de l'INSEE. Volumétrie : plusieurs millions de transactions DVF, ~32 000 communes. »

Slide 4 — Architecture technique & traitement des données : médaillon + orchestration (1:45)

À l'écran - Schéma **Bronze** → **Silver** → **Gold** (réutiliser docs/diagrams/data_flow_archi.png). - **Bronze** : brut sur object store S3 · **Silver** : nettoyage **Spark** → Parquet · **Gold** : agrégats commune × année dans

PostgreSQL, servis par l'API. - **Airflow** : 3 DAGs mensuels (housing / mobility / amenities), jobs Spark sérialisés.

À l'oral

« Côté data, **architecture médaillon**. **Bronze** : la donnée brute telle quelle sur S3, une source de vérité rejeuable. **Silver** : Spark nettoie, filtre les aberrations, normalise, et écrit du Parquet. **Gold** : on **pré-calcule** des agrégats par commune et par année dans PostgreSQL, pour que l'API réponde en quelques millisecondes. Le tout est orchestré par **Airflow**, un DAG par domaine, planifié chaque mois, avec les jobs Spark exécutés **un à un** pour ne pas saturer la machine de calcul. »

Slide 5 — Déploiement / infra prod (1:30)

À l'écran - **CI/CD GitHub Actions** → build + push de **4 images** sur GHCR (api, front, spark, airflow). - **VPS** (plan de service) : Caddy HTTPS, API, front, Redis, Airflow. - **VM GCP** (plan de données, **à la demande**) : Airflow **allume** → Spark → **éteint**. - *Schéma* : *GitHub* → *GHCR* → *VPS* ; *Airflow(VPS)* ↔ *VM GCP*.

À l'oral

« En production, tout est conteneurisé et déployé par **CI/CD** : à chaque push, GitHub Actions construit nos 4 images et les publie. Un **VPS** héberge le plan de service — API, front, Airflow — en HTTPS. Et pour le calcul lourd, une **VM Spark sur GCP qu'Airflow allume avant chaque run et éteint après** : on ne paie la puissance que quand on l'utilise. C'est un vrai découplage **service / calcul**. »

Slide 6 — Gouvernance & qualité des données (1:30)

À l'écran - **OpenMetadata** : catalogue + **lignage** des jeux de données. - **Portes qualité** dans les DAGs (règles YAML : complétude, bornes, fraîcheur) — un contrôle qui casse **fait échouer** le pipeline. - Secrets hors du code (`.env.prod` / secrets GitHub, gitignore).

À l'oral

« On a pris la gouvernance au sérieux. **OpenMetadata** catalogue nos datasets et trace le **lignage** source → Silver → Gold. Chaque pipeline embarque des **portes qualité** : des règles déclaratives en YAML vérifient la complétude, les plages de valeurs, la fraîcheur — et si une règle critique casse, le run **échoue** au lieu de publier de la donnée fausse. Les secrets ne sont jamais dans le code. »

Slide 7 — Stack technique & choix (1:30)

À l'écran - Spark 3.5 · MinIO/S3 · PostgreSQL 16 · Redis · FastAPI · Next.js 16 · Airflow 2.10 · OpenMetadata - **Choix assumé** : PostgreSQL en Gold (pas ClickHouse) — Gold **pré-agrégé**, relationnel + JSON suffisent, simplicité d'exploitation.

À l'oral

« La stack : Spark pour le traitement distribué, S3 pour le stockage, PostgreSQL pour le Gold servi, Redis en cache, FastAPI et Next.js pour l'app, Airflow pour orchestrer, OpenMetadata pour la gouvernance. Un choix qu'on assume : **PostgreSQL plutôt qu'une base OLAP** comme ClickHouse — notre Gold est déjà pré-agrégé et tient largement dans du relationnel. On a préféré la simplicité d'exploitation à une brique de plus. Voilà pour la fabrication ; [nom] va vous montrer le métier et la démo. »

PARTIE 2 — Le métier & la démo (10 min)

Slide 8 — Zoom métier : le score de correspondance (1:45)

À l'écran - Score **0-100** = moyenne **pondérée** de 4 notes : budget / transport / services / jeunes actifs. - Poids réglés par **l'utilisateur** (curseurs) ; dimensions sans donnée **exclues**. - Exemple chiffré : $(80 \cdot 60 + 80 \cdot 50 + 75 \cdot 40 + 60 \cdot 30) / 180 \approx 76/100 \rightarrow$ commune verte.

À l'oral

« Le cœur produit, c'est le **score de correspondance**. On note chaque commune sur 4 axes, entre 0 et 100 : le budget, le transport, les services, et l'ambiance jeune. Puis on fait une **moyenne pondérée par les curseurs** de l'utilisateur — c'est ça qui rend la carte personnelle : "*le budget compte plus que le transport*", et tout se recolore. Et si une donnée manque, on **exclut** la dimension au lieu de pénaliser la commune. »

Slide 9 — Zoom métier : des règles justifiées (1:30)

À l'écran - **Médiane** (pas moyenne) — distribution asymétrique (ventes de luxe). - Bornes **[500 ; 20 000] €/m²** — anti-aberrations DVF, stables & reproductibles. - **Années de revenu** = $\text{prix_m2} \times 70 / \text{revenu_median}$ — sans hypothèse de taux. - Analyse textuelle par **règles** (déterministe) plutôt que **LLM**.

À l'oral

« Et derrière chaque métrique, un **choix justifié**, pas arbitraire. La **médiane** plutôt que la moyenne, parce que les prix sont tirés par quelques ventes de luxe. Des **bornes fixes** pour écarter les saisies aberrantes de DVF de façon reproductible. L'abordabilité en **années de revenu** : intuitif, sans hypothèse de taux d'intérêt. Et l'analyse écrite est générée par un **moteur à règles** — déterministe et auditable — pas par un LLM, justement pour être **défendable**. Tout est détaillé dans notre rapport. »

Slide 10 — DÉMO LIVE (3:30)

Déroulé (voir la checklist en fin de document) 1. Régler les curseurs → la carte **se recolore**. 2. Basculer **France** ↔ **Île-de-France**. 3. Cliquer une commune → **fiche** (score, *surface atteignable*, tendance). 4. Page **Statistiques** (1-2 graphiques marquants) + **Analyse** textuelle. 5. (*si le temps*) **Tableau** triable + export CSV.

À l'oral

« Place à la démo. Carte de France... je dis que le **budget** prime... la carte se recolore. Je bascule sur l'Île-de-France. Je clique une commune : son score détaillé, et surtout "*avec votre budget, ici, vous pourriez acheter X m²*". Voici les statistiques de la zone, et l'analyse **rédigée automatiquement**. »

Slide 11 — Bilan & difficultés (1:15)

À l'écran - Fait : médaillon complet, 4 domaines, app publique, gouvernance, prod CI/CD. - **Galères clés** : registre GHCR privé (org sans admin), clés de service GCP interdites → auth utilisateur, IP VM changeante → orchestration par nom, VM saturée → jobs Spark sérialisés. - **Perspectives** : prédiction de prix, plus de villes, alerting qualité.

À l'oral

« Pour conclure : on a une chaîne data **de bout en bout**, de l'ingestion à une app publique, avec gouvernance et déploiement automatisé. On ne cache pas les galères — le registre d'images privé sans droits admin, les clés GCP interdites par une policy, l'IP de la VM qui changeait, la VM saturée par des jobs parallèles : chacune nous a appris l'**exploitation réelle**. Et pour la suite : de la prédiction de prix et plus de villes. Merci, on est à vous pour les questions. »

Slide 12 — Questions (2:00)

À l'écran - Merci ! Questions ? - (lien démo · repo · rapport)

Checklist démo (à préparer AVANT de passer)

- [] L'app est **déjà ouverte** et connectée à l'API (pas de « Données mockées »).
 - [] Onglet de secours : **captures / vidéo** de la démo si le live plante.
 - [] Une commune « qui parle » repérée d'avance (ex. Paris + une commune abordable).
 - [] Zoom navigateur réglé, curseur visible, notifications coupées.
 - [] Airflow ouvert sur un **run réussi** du DAG (si on demande à voir un pipeline).
-

Questions probables du jury (+ réponses courtes)

« Pourquoi PostgreSQL et pas une base OLAP (ClickHouse) ? »

Le Gold est déjà **pré-agrégé** (commune × année) : volumétrie modeste, requêtes simples. PostgreSQL suffit et évite une brique d'exploitation. ClickHouse serait justifié pour des milliards de lignes non agrégées.

« Pourquoi la médiane et pas la moyenne ? »

Distribution **asymétrique à droite** (ventes de luxe) : la médiane reflète la transaction typique ; la moyenne gonflerait le prix « normal ».

« Pourquoi un moteur à règles et pas un LLM pour l'analyse ? »

Déterministe, explicable, sans hallucination ni coût d'API. Chaque phrase doit être justifiable ; un LLM pourrait inventer des chiffres.

« Comment gérez-vous la volumétrie / la montée en charge ? »

Spark en distribué pour le Silver ; Gold **pré-agrégé** donc léger ; Redis en cache API ; VM de calcul **dimensionnée à la demande**.

« Qualité des données : comment savez-vous que c'est juste ? »

Bornes anti-aberrantes, filtre mono-bien sur DVF, et **portes qualité YAML** dans les DAGs (complétude, plages, fraîcheur) qui **font échouer** le run.

« Pourquoi une VM à la demande plutôt qu'un cluster permanent ? »

Coût. Batch mensuel : inutile de payer 24/7. Airflow allume, lance Spark, éteint.

« Que se passe-t-il si un pipeline échoue la nuit ? »

Retries Airflow, VM **éteinte quoi qu'il arrive** (`trigger_rule=all_done`), portes qualité qui empêchent de publier de la donnée corrompue.

« RGPD / données personnelles ? »

Uniquement des **données ouvertes agrégées** à la commune, rien de nominatif — pas de sujet RGPD côté data. Les comptes de l'app sont gérés à part.

Conseils de prise de parole

- **Une idée par slide.** Le texte du slide = repères ; le script se **dit**.
- Annoncez la **passation** entre partie 1 et partie 2 (« [nom] prend la suite pour le métier et la démo »).

- Sur la démo : **montrez la valeur** (« avec votre budget, ici, X m² »), pas les menus.
- Gardez ~2 min pour les questions ; si vous débordez, **sautez le Tableau** en démo.
- Terminez par une phrase forte, pas par « voilà, c'est tout ».